COLLEGE OF CREATIVE INNOVATION

Master of Software Engineering — Level 8

MSE802 Quantum Computing

Assessment 2 — Quantum Project (60%)

Task 4 Report: Machine Learning Quantum Analysis

Bars-vs-Stripes Quantum Classifier: Circuit Analysis, Iteration Monitoring and Local Comparison

GROUP PROJECT

Student 1	Anish Shrestha — Student ID 270889991
Student 2	Sandhya Tiwari — Student ID 270918511
Lecturer	Dr Helena Bahrami
Course	MSE802 Quantum Computing (15 credits)
Date	4 November 2026
GitHub repository	github.com/anishnz/quantumcomputing_YB_Anish
Referencing style	APA 7th edition

Auckland, New Zealand

Declaration of Originality

We, the members of this group named below, declare that this report and the accompanying code are our own group work, completed for MSE802 Assessment 2. All sources of information, code and ideas from others, including the lecture notes and the supplied lab notebooks, have been acknowledged and referenced in APA 7th style. This work has not been submitted for any other course. We understand Yoobee College's policies on plagiarism and collusion as described in the Student Handbook.

We confirm that the results reported here come from the notebooks submitted with this report. Results produced on a local simulator because the Quokka device was unavailable are clearly labelled as such and are not presented as device measurements.

Use of digital tools: Python libraries (Cirq, Qiskit, NumPy, Matplotlib) were used for all computation. A generative AI assistant was used to support code drafting, debugging and language editing; all code was run, checked and understood by the group, and the analysis and conclusions are our own. [Edit this sentence to match Yoobee's AI-use policy and your actual use.]

Name	Student ID
Anish Shrestha	270889991
Sandhya Tiwari	270918511

Project Links and How to Run Online

Every result in this document can be reproduced online, with nothing to install. Copy a link below and paste it into the address bar of a web browser.

GitHub repository (all code, results and reports):
https://github.com/anishnz/quantumcomputing_YB_Anish

Google Colab – run ALL tasks (recommended):
https://colab.research.google.com/github/anishnz/quantumcomputing_YB_Anish/blob/main/Assignment_2_Quantum/Colab_Run_All_Tasks.ipynb

Google Colab – Task 1 (Bell state, Cirq):
https://colab.research.google.com/github/anishnz/quantumcomputing_YB_Anish/blob/main/Assignment_2_Quantum/Task_1_Bell_State/Task1_Bell_State_Cirq_Quokka.ipynb

Google Colab – Task 2 (Qiskit circuit + QASM):
https://colab.research.google.com/github/anishnz/quantumcomputing_YB_Anish/blob/main/Assignment_2_Quantum/Task_2_Qiskit_QASM/Task2_Qiskit_Circuits_and_QASM.ipynb

Google Colab – Task 3 (quantum tic-tac-toe):
https://colab.research.google.com/github/anishnz/quantumcomputing_YB_Anish/blob/main/Assignment_2_Quantum/Task_3_Quantum_TicTacToe/Task3_Quantum_TicTacToe_Completed.ipynb

Google Colab – Task 4 (quantum machine learning):
https://colab.research.google.com/github/anishnz/quantumcomputing_YB_Anish/blob/main/Assignment_2_Quantum/Task_4_Quantum_ML/Task4_Quantum_ML_Modified.ipynb

Download the whole project as a ZIP:
https://github.com/anishnz/quantumcomputing_YB_Anish/archive/refs/heads/main.zip

How to run the project online in Google Colab (step by step)

1. Copy the link "Google Colab – run ALL tasks" above and paste it into the address bar of Chrome, Edge or Safari, then press Enter.
2. Sign in with any Google account if Colab asks you to (Colab is free).
3. Click Runtime → Run all. If Colab shows "Warning: This notebook was not authored by Google", click Run anyway.
4. Wait about 15–20 minutes. The notebook downloads the GitHub repository, installs Cirq and Qiskit, checks which Quokka devices are online, runs Tasks 1–4 with 1000 shots (Task 4 sends 900 circuits to Quokka), shows every table and chart, runs the laptop-vs-Quokka comparison and the alternative code.
5. Read the results under each step. Every result says where it ran: QUOKKA DEVICE (with the device name, e.g. quokka2) or LOCAL SIMULATOR if no Quokka device is online.
6. To keep your own copy, click File → Save a copy in Drive. Step 7 of the notebook can also download all outputs as a ZIP.
7. To run only one task, open that task's Colab link above, then click Runtime → Run all. Its first code cell downloads the project automatically.
8. To run on your own computer instead: download the ZIP link above, unzip it, open the Assignment_2_Quantum folder and double-click run_project.bat (Windows) or run_project.command (macOS). Python 3.12 or newer is needed.

Table of Contents

Declaration of Originality..... 1
Project Links and How to Run Online..... 2
Table of Contents..... 3
1. Task 4 – Machine Learning Quantum Analysis..... 4
1.1 What problem does the code solve? (Problem formulation)..... 4
1.2 Which part of the code is the input to the circuit?..... 4
1.3 How does the circuit work? (Quantum part, excluding the optimiser)..... 4
1.4 What did we add to monitor every iteration?..... 5
1.5 What do the two required graphs show?..... 6
1.6 How did we make the code run without the quantum computer or circuit?..... 7
1.7 What were the differences in efficiency and effectiveness?..... 7
1.8 Did anything not work as expected?..... 9
1.9 What is our overall judgement for Task 4?..... 9
References..... 9

(If page numbers are missing, right-click the table and choose Update Field.)

> This is the separate Task 4 report requested by the brief. Its content is identical to Section 6 of the group report Assignment2.docx, which also contains the introduction, the other tasks and the full discussion.

1. Task 4 – Machine Learning Quantum Analysis

> Task question: Analyse the code (circuit and problem formulation; data input; quantum functionality, excluding the optimiser); execute it, monitor the output at each iteration and plot iteration vs output and iteration vs time; modify it to run locally without the quantum computer or circuit and document differences in efficiency and effectiveness; submit the modified .ipynb and a report. Notebook: Task_4_Quantum_ML/Task4_Quantum_ML_Modified.ipynb (original kept beside it)

1.1 What problem does the code solve? (Problem formulation)

The supplied Lab 10 notebook (Bahrami, 2026c) trains a quantum circuit to classify tiny 2 × 2 black-and-white images into two classes, which the code calls **stripes** (label 0) and **bars** (label 1). `generate_data(2)` creates all line patterns and removes the all-black and all-white images, because they belong to both classes. Four images remain: [1,0,1,0] and [0,1,0,1] (label 0), and [1,1,0,0] and [0,0,1,1] (label 1). Three are used for training and one for testing (75/25 split).

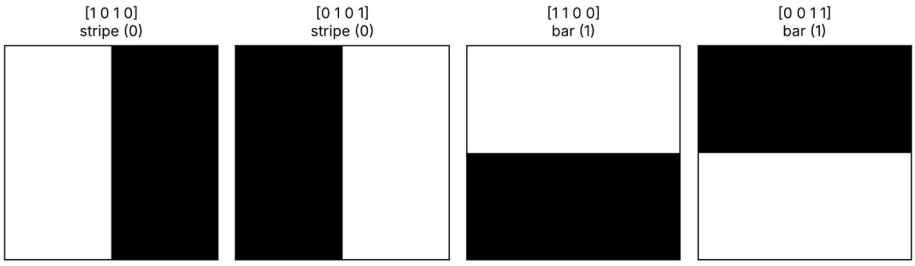


Figure 1. *The four images, their pixel vectors and labels*

The model output is the probability that the last qubit is measured as 1, estimated from shots: $\hat{y}(\theta, x) \approx$ (number of 1s) / shots. The **loss** minimised is the mean absolute error over the training images:

$$L(\theta) = (1/N)\ \Sigma_n \mid \hat{y}(\theta, x_n) - y_n \mid$$

A good model gives $\hat{y} \to 1$ for bars and $\hat{y} \to 0$ for stripes; an image is classified as a bar when $\hat{y} > 0.5$.

1.2 Which part of the code is the input to the circuit?

The input is the loop at the top of `create_circuit`: for every pixel i, `if data[i] > 0.5`, an **X gate** is added to qubit i. Each pixel is one qubit, and a white pixel flips its qubit from $|0\rangle$ to $|1\rangle$. This is **basis encoding**: the image becomes the starting basis state, e.g. [1,0,1,0] → $|q3q2q1q0\rangle = |0101\rangle$ (Schuld & Petruccione, 2021). The data enters only here; everything after it is the trainable model.

1.3 How does the circuit work? (Quantum part, excluding the optimiser)

The trainable part is built from **blocks**. `block(angles, qubits)` applies $R_Y(\theta_a)$ to one qubit, $R_Y(\theta_b)$ to another, and then a CNOT between them. `generate_pairs` splits the qubits in half again and again and joins the halves, giving the pairs (0,1), (2,3) and then (1,3). Information from all four pixels therefore flows into **qubit 3** in only $\log_2 4 = 2$ layers. This is a **tree tensor network (TTN)** classifier of the kind studied by Grant et al. (2018) and Huggins et al. (2019). With 3 blocks × 2 angles there are **6 trainable parameters**, and only qubit 3 is measured.

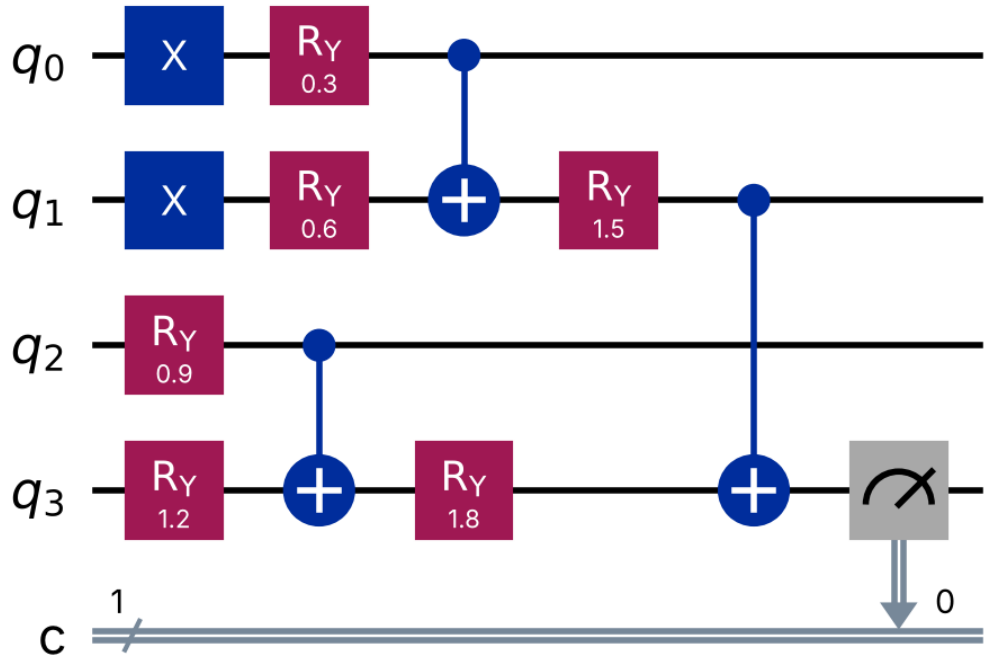


Figure 2. *Classifier circuit for image [1,1,0,0]: X gates encode the pixels, then three RY–RY–CNOT blocks form a tree into q3*

The RY gates give each block adjustable 'weights', while the CNOTs make the output depend on *combinations* of pixels. If a control qubit is in $|1\rangle$, the CNOT flips the target, so the final value of q3 behaves like an XOR (parity) of pixels that is smoothly adjusted by the angles. This structure turns out to be the key to the results below.

1.4 What did we add to monitor every iteration?

We modified the SPSA optimiser (Spall, 1992) without changing its algorithm. SPSA estimates the gradient from just **two** loss evaluations per iteration, whatever the number of parameters, which suits noisy, expensive quantum evaluations. We added a timer around each update and, after each update, one extra evaluation of the training loss, the **output measure**. This monitoring evaluation is timed separately so it does not inflate the iteration time. Settings were kept as in the original ($a = 1$, $c = 0.5$, $\alpha = 0.602$, $\gamma = 0.101$, 100 iterations), except that the shots per circuit were raised from 100 to the group's agreed **1000**, and a fixed seed (2026) makes runs repeatable. Each iteration runs 3 evaluations × 3 training images = 9 circuits, so the run executed **900 circuits** in total.

1.5 What do the two required graphs show?

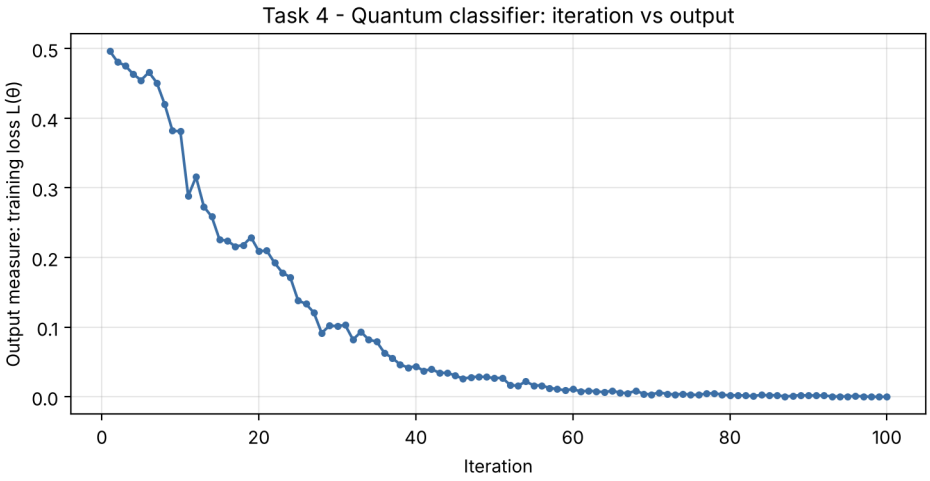


Figure 3. *Required graph 1: iteration number versus output measure (training loss)*

Table 1. *Excerpt from the 100-iteration log (task4_quantum_iterations.csv)*

Iteration	Output (training loss)	Note
1	0.497	random start: no better than guessing
10	0.381	
20	0.209	
30	0.102	
38	0.046	first iteration below 0.05
50	0.027	
70	0.003	
100	0.000	end of training

Graph 1 (run on the Quokka device (quokka2)) shows the loss falling from about 0.50, which is equal to random guessing, to 0.000 by iteration 100. It fell quickly in the first 40 iterations and then flattened. The curve is not perfectly smooth. Each loss value is itself an estimate from 1000 shots per image, and SPSA's random ±1 directions sometimes give a slightly worse step (e.g. iterations 10–11 and 15–20). The step size a_k = 1/(k+1)^0.602 shrinks over time, which explains the slow final convergence.

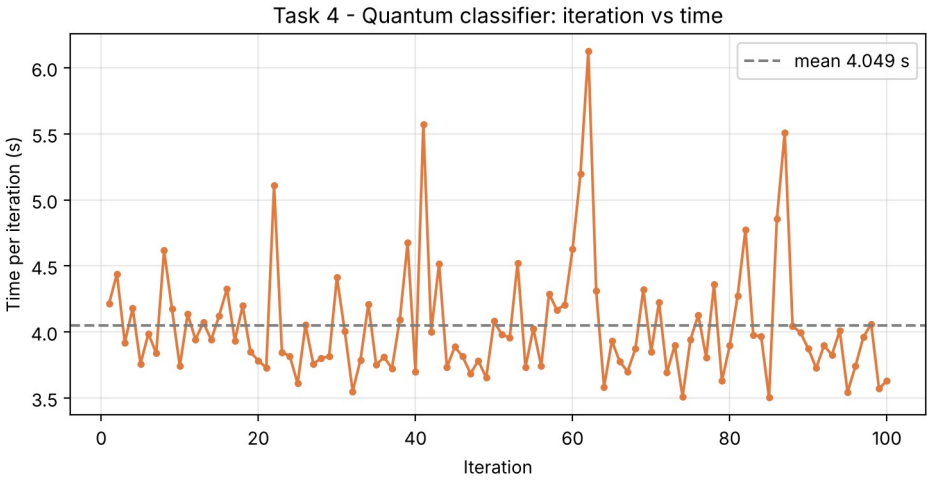


Figure 4. *Required graph 2: iteration number versus time per iteration*

Graph 2 shows an almost **constant** time per iteration of **4.05 s** on average (range 3.50–6.13 s). This is expected: every iteration runs the same number of circuits (6 for the update) with the same size and shot count, so the cost does not depend on how good the model is. Because the circuits ran on Quokka, almost all of this time is the internet round trip for each of the six requests; Quokka itself needs only a few milliseconds per circuit (Section 7). The spikes are slower network responses.

1.6 How did we make the code run without the quantum computer or circuit?

The brief can be read in two ways, so we built **both** and added one fairer baseline. Every version uses the same loss, the same SPSA settings and the same random seed, so only the model changes:

- **Local-A: no quantum computer.** The same circuit is simulated exactly with our own NumPy state-vector code: 16 complex amplitudes, with gates applied as matrices. It computes P(q3 = 1) exactly, so there is no shot noise. It was verified against Qiskit (both gave 0.500625 for a random test case).
- **Local-B: no quantum circuit.** A classical logistic regression $\hat{y} = \sigma(w \cdot x + b)$ with 5 parameters, close to the circuit's 6.
- **Local-C: a fair classical baseline.** Logistic regression with pairwise pixel products added (4 + 6 features, 11 parameters). This is the standard classical way to model interactions.

1.7 What were the differences in efficiency and effectiveness?

Table 2. *Quantum circuit versus local versions (100 SPSA iterations, seed 2026). Accuracy uses each trained model's exact output; times vary slightly between runs*

Version	Total time (s)	Time / iteration (s)	Circuit runs	Final loss	Test acc.	All 4 images
Quantum circuit (1000 shots, Quokka quokka2)	606.2	4.049	900	0.000	1/1	4/4
Local-A: same circuit, exact NumPy	0.18	0.0012	0	0.0003	1/1	4/4
Local-B: classical linear	0.007	0.00005	0	0.296	0/1	3/4
Local-C: classical + pairwise features	0.026	0.00019	0	0.206	0/1	3/4

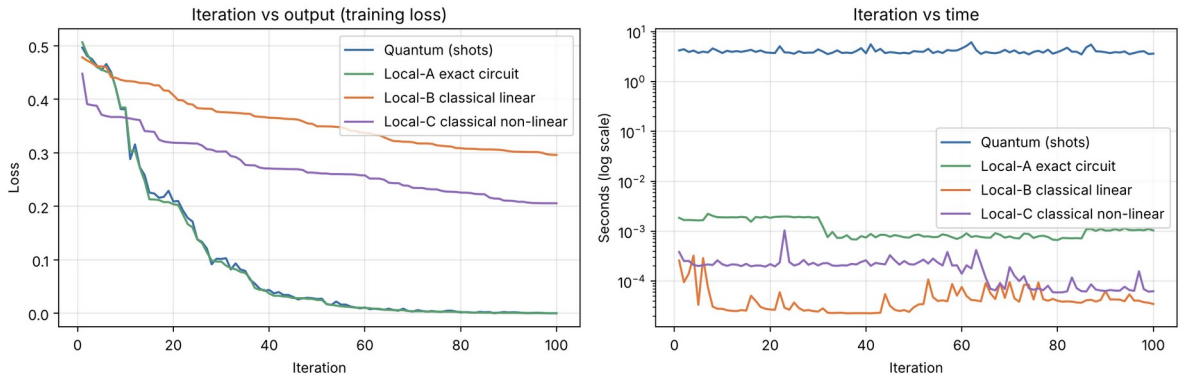


Figure 5. *All versions: loss per iteration (left) and time per iteration on a log scale (right)*

Efficiency. Removing the quantum device made each iteration about **3490 times faster** (4.049 s → 0.0012 s). A shot-based run needs 1000 samples per circuit, QASM parsing and, on Quokka, one network request per circuit (900 in total), whereas the exact simulation needs one small matrix calculation. Local-A and the quantum run followed almost the same loss curve, so the shot noise slowed the computer, not the learning. This advantage disappears as problems grow: exact simulation needs 2^n amplitudes, so 50 qubits would need about 10^{15} complex numbers, which is why real quantum hardware becomes useful only at scale (Section 7.4 measures where this begins for Quokka).

Effectiveness. The circuit (both quantum and Local-A) reached zero training loss and also classified the **unseen test image** correctly. Both classical models fitted the training images but **failed the test image**, and their losses stayed high (0.30 and 0.21). The reason is the shape of the data. In each class the two images are exact opposites, so the class sums are identical and **no straight line can separate the classes**. Like XOR, this is the classic limit of linear models (Minsky & Papert, 1969). In fact, the label equals $\mathbf{x_1}$ **XOR** $\mathbf{x_3}$ (Figure 6).

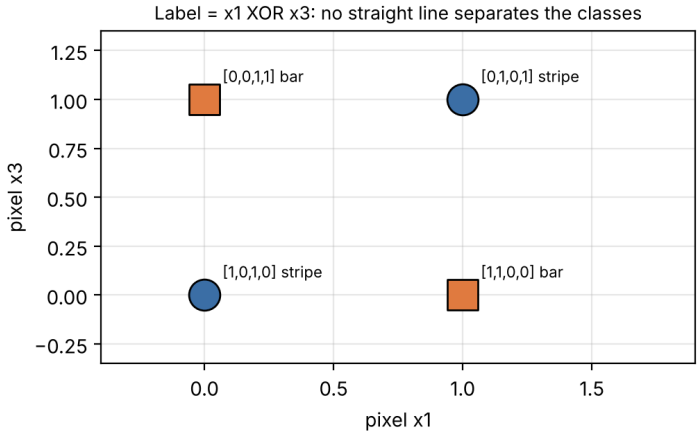


Figure 6. *The data is XOR-shaped: label = x1 XOR x3, so a linear classifier cannot separate it*

Local-C has the right *kind* of features, but the test image's key feature (the product $x_0 \cdot x_1$) is zero in all three training images, so its weight is never learned. The circuit generalised because its CNOT tree computes parity naturally. Its built-in bias (an XOR-like structure) happens to match this problem. This is an honest, limited claim: with one test image the test score is weak evidence, and a classical model designed with XOR features would also succeed. The fair conclusion is that the circuit's **structure**, not quantum speed, made it effective here.

1.8 Did anything not work as expected?

Yes. The original warm-up (one qubit learning to output 1 with Nelder–Mead, 10 iterations) **failed** in our run: on Quokka it stopped at θ = −0.43 with loss 0.89 instead of θ = π (the earlier simulator run stopped at θ = −0.47 with loss 0.87). Near θ = 0 the loss $\cos^4(\theta/2)$ is almost flat, and differences between nearby points are smaller than the shot noise, so the simplex method (Nelder & Mead, 1965) could not tell which way to move in 10 iterations. This small failure illustrates a real problem in quantum machine learning: noise plus flat loss landscapes make training hard, and in large circuits this becomes the 'barren plateau' problem (McClean et al., 2018). SPSA was more robust because it uses large, random perturbations.

1.9 What is our overall judgement for Task 4?

- **Input:** pixels → X gates (basis encoding) in `create_circuit`; **model:** 6 RY angles in a TTN of RY–RY–CNOT blocks; **output:** P(q3 = 1).
- **Monitoring:** on the Quokka device (quokka2) the loss fell from 0.497 to 0.000 in 100 iterations; time per iteration was flat at about 4.05 s.
- **Efficiency:** at 4 qubits, simulation without the device is about 3490 times faster; real devices only pay off where simulation is impossible.
- **Effectiveness:** the circuit solved an XOR-type problem that linear classical models could not. This comes from its CNOT structure, not from a proven quantum advantage (Benedetti et al., 2019).

References

Bahrami, H. (2026c). *Lab 10 — Quantum ML* [Unpublished Jupyter notebook, Quantum_ML_AS2.ipynb]. MSE802 Quantum Computing, Yoobee College of Creative Innovation.

Benedetti, M., Lloyd, E., Sack, S., & Fiorentini, M. (2019). Parameterized quantum circuits as machine learning models. *Quantum Science and Technology*, *4*(4), Article 043001. https://doi.org/10.1088/2058-9565/ab4eb5

Grant, E., Benedetti, M., Cao, S., Hallam, A., Lockhart, J., Stojevic, V., Green, A. G., & Severini, S. (2018). Hierarchical quantum classifiers. *npj Quantum Information*, *4*, Article 65. https://doi.org/10.1038/s41534-018-0116-9

Huggins, W., Patil, P., Mitchell, B., Whaley, K. B., & Stoudenmire, E. M. (2019). Towards quantum machine learning with tensor networks. *Quantum Science and Technology*, *4*(2), Article 024001. https://doi.org/10.1088/2058-9565/aaea94

McClean, J. R., Boixo, S., Smelyanskiy, V. N., Babbush, R., & Neven, H. (2018). Barren plateaus in quantum neural network training landscapes. *Nature Communications*, *9*, Article 4812. https://doi.org/10.1038/s41467-018-07090-4

Minsky, M., & Papert, S. (1969). *Perceptrons: An introduction to computational geometry*. MIT Press.

Nelder, J. A., & Mead, R. (1965). A simplex method for function minimization. *The Computer Journal*, *7*(4), 308–313. https://doi.org/10.1093/comjnl/7.4.308

Schuld, M., & Petruccione, F. (2021). *Machine learning with quantum computers* (2nd ed.). Springer. https://doi.org/10.1007/978-3-030-83098-4

Spall, J. C. (1992). Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. *IEEE Transactions on Automatic Control*, *37*(3), 332–341. https://doi.org/10.1109/9.119632